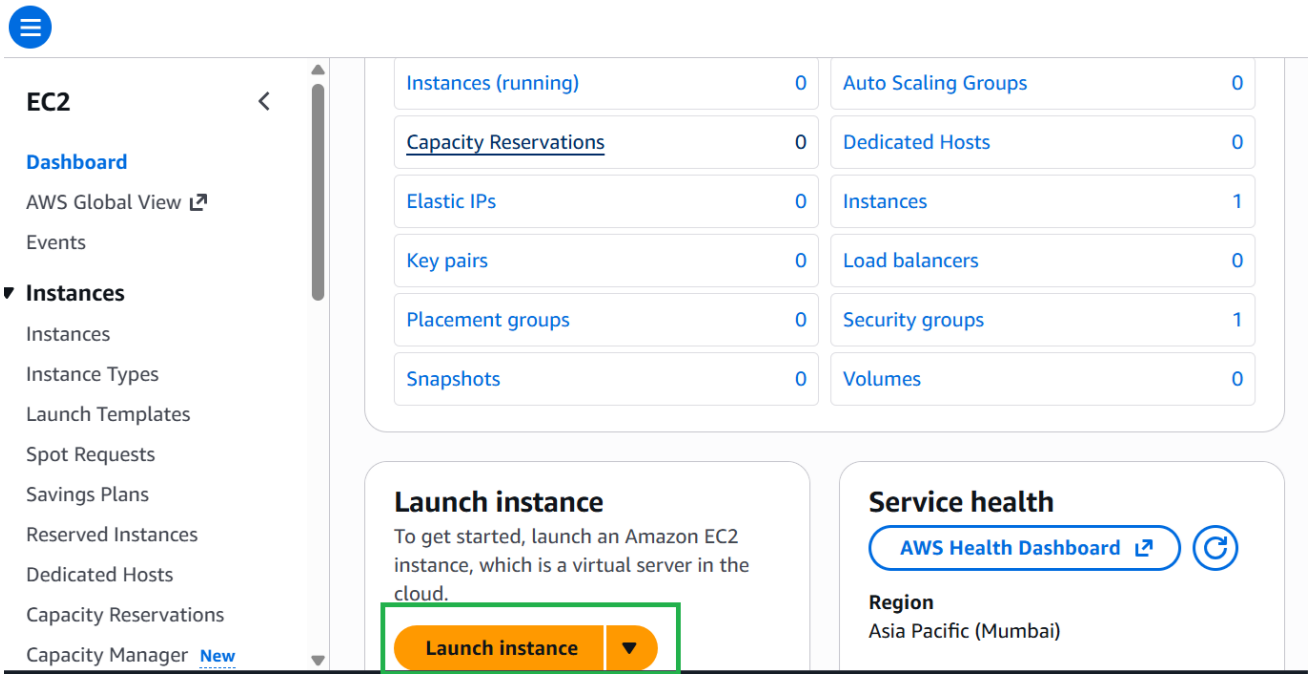
# EC2 Steps

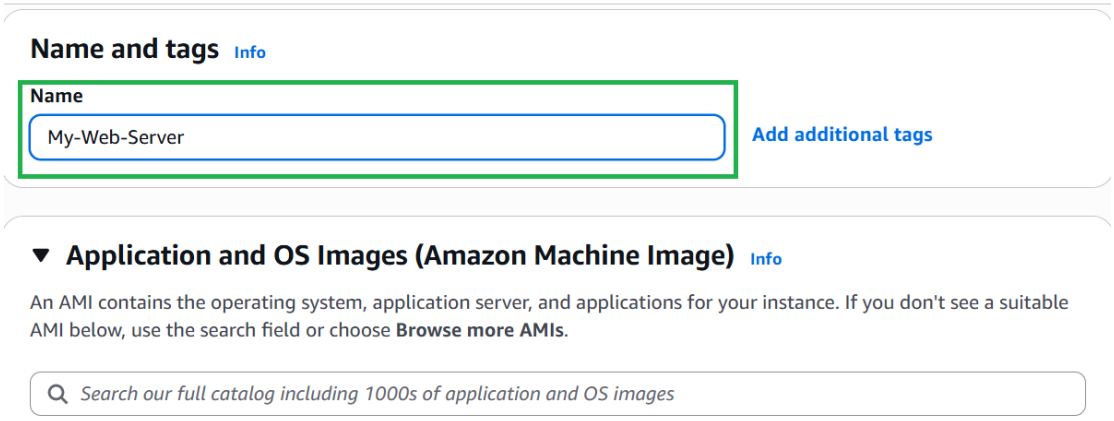
**Launch an Instance** console.




## Step 2

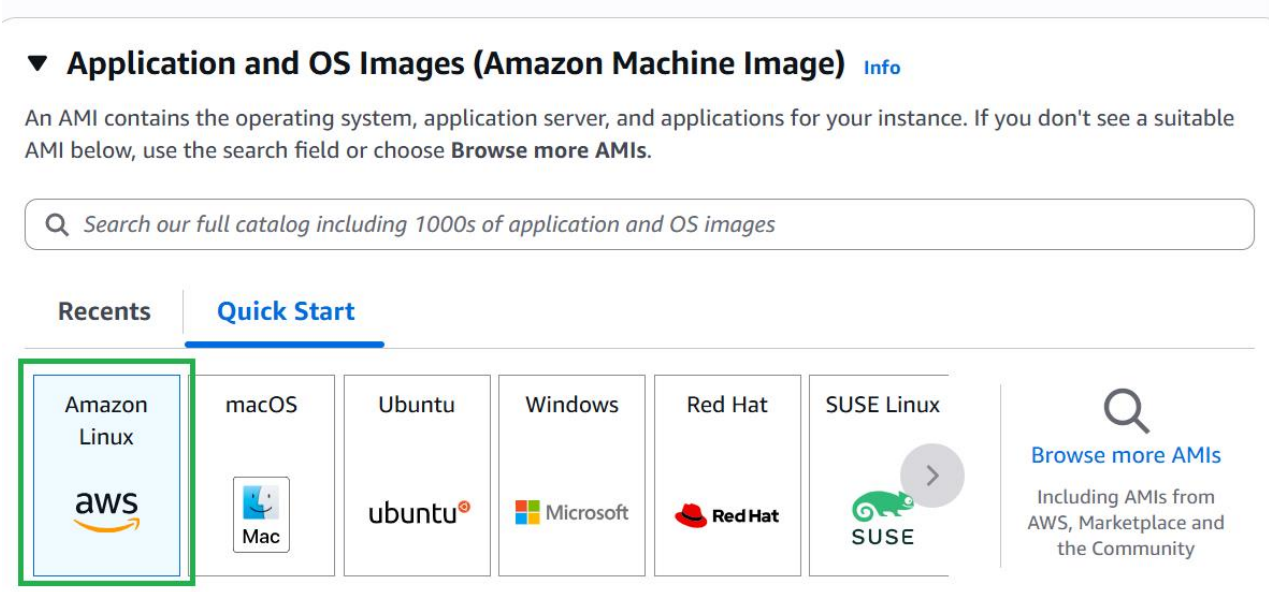
### A) Name and Tags

- **Suggested Name:** `My-Web-Server` or `Dev-Environment-01`.
- **Why:** Giving it a clear name helps you identify it later in the dashboard, especially when you have multiple instances running.



## B) Application and OS Images (AMI)

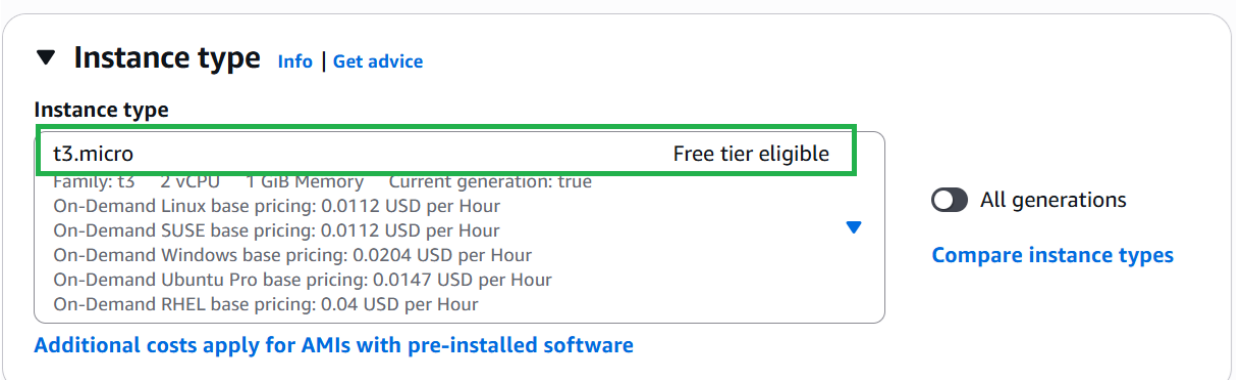
- **Selection:** Keep **Amazon Linux 2023** (which you currently have selected).
- **Why:** It is optimized for AWS, highly secure, and is **Free Tier eligible**. It also uses the `ec2-user` username, which is standard for most AWS tutorials.




## C) Instance Type

**Selection:** `t3.micro` (or `t2.micro` if available).

**Why:** You already have `t3.micro` selected, which is excellent. It is **Free Tier eligible** (depending on your region/account age) and provides 2 vCPUs and 1 GiB of RAM, which is plenty for learning or small applications.




## D) Key Pair (Login)

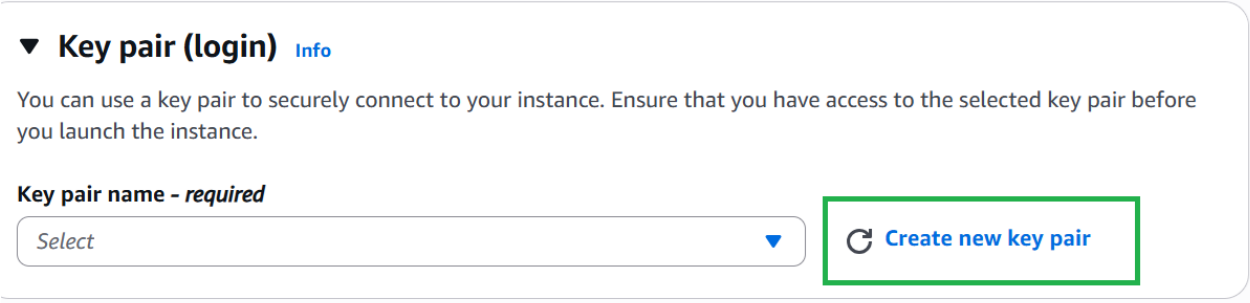
**Selection:** Click **Create new key pair**.

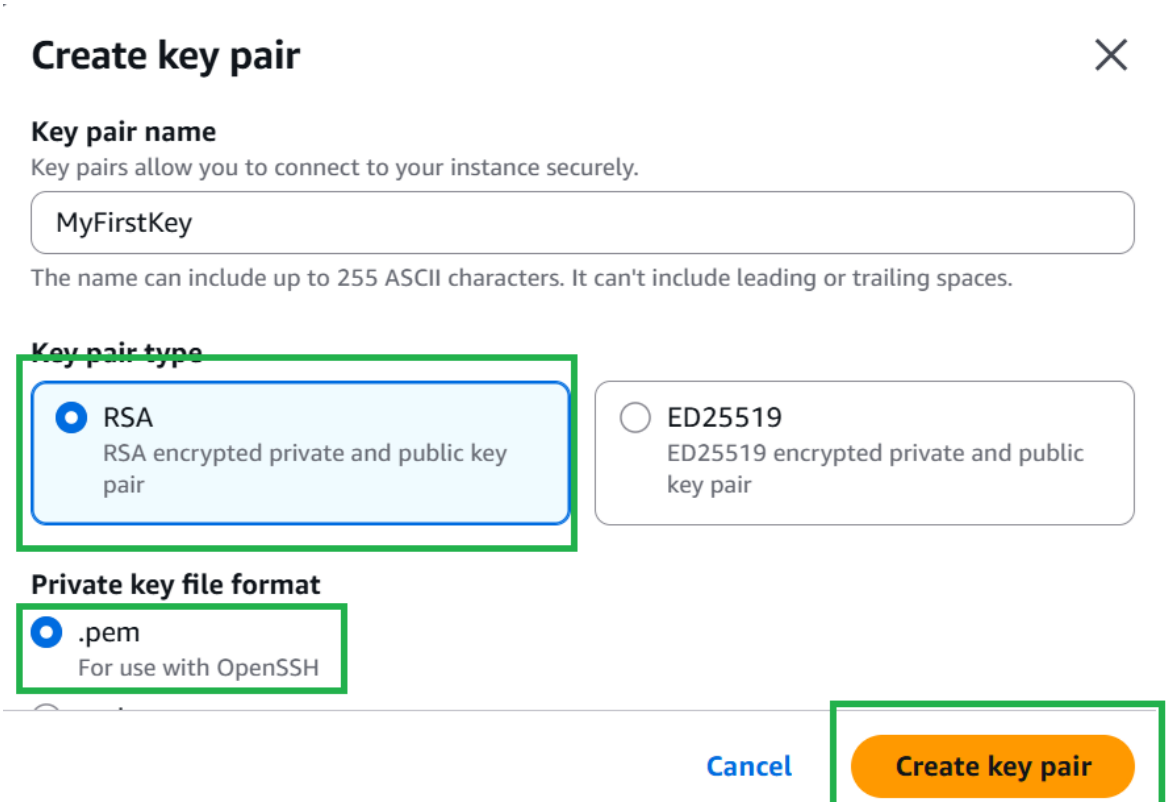
**Settings:**

a. **Key pair name:** `MyFirstKey`

b. **Key pair type:** RSA
c. **Private key file format:** `.pem` (for OpenSSH/Mac/Linux/Windows 10+) or `.ppk` (if you specifically use the older PuTTY tool).

**Action: Download and save this immediately.** You cannot download it again once the instance is created.



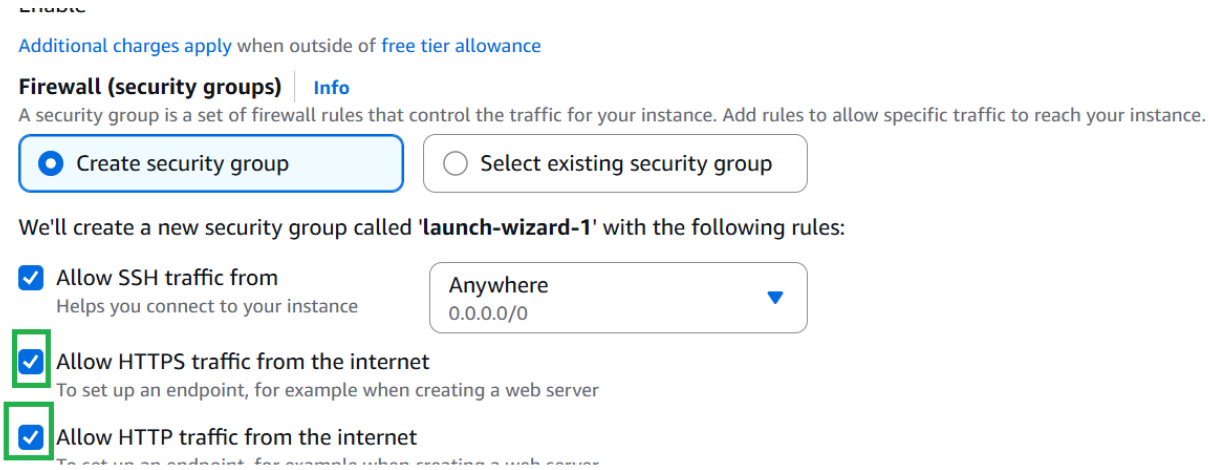

E) **Network Settings (Firewall)**

**Allow SSH traffic from:** Change "Anywhere" to **My IP**.

a. *Security Tip:* Leaving this at `0.0.0.0/0` (Anywhere) allows the whole world to try and brute-force your login. Selecting "My IP" locks it down to just your computer.

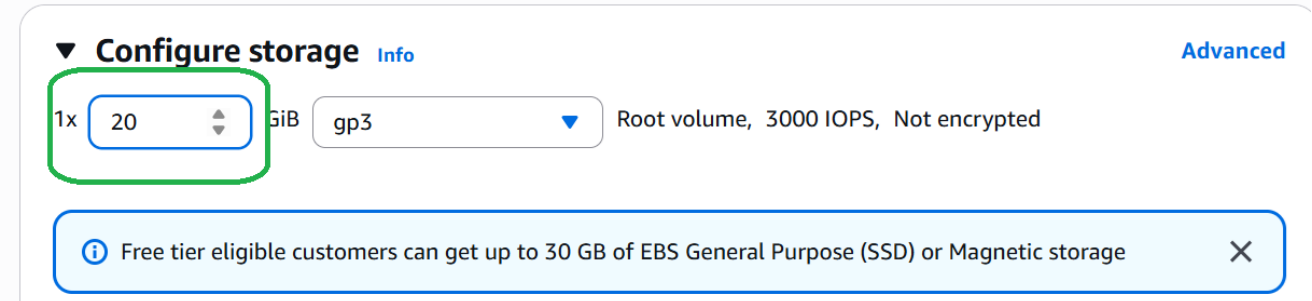
**Allow HTTPS/HTTP traffic:** Check **both boxes** if you plan on hosting a website. If you are just using this f

or backend practice, you can leave them unchecked for now.



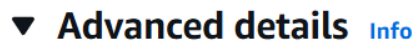

## F) Configure Storage

- **Selection:** Change the size to **20 GiB** or **30 GiB**.
- **Volume Type:** Keep **gp3**.
- **Why:** AWS Free Tier gives you up to 30 GB of EBS storage. Using 20 or 30 GB gives you more breathing room than the default 8 GB without costing extra money.




## G) Advanced Details (Optional but Useful)

If you want the server to do something immediately upon startup, scroll to the bottom and paste this into the **User Data** field:



**User data - *optional*** | Info

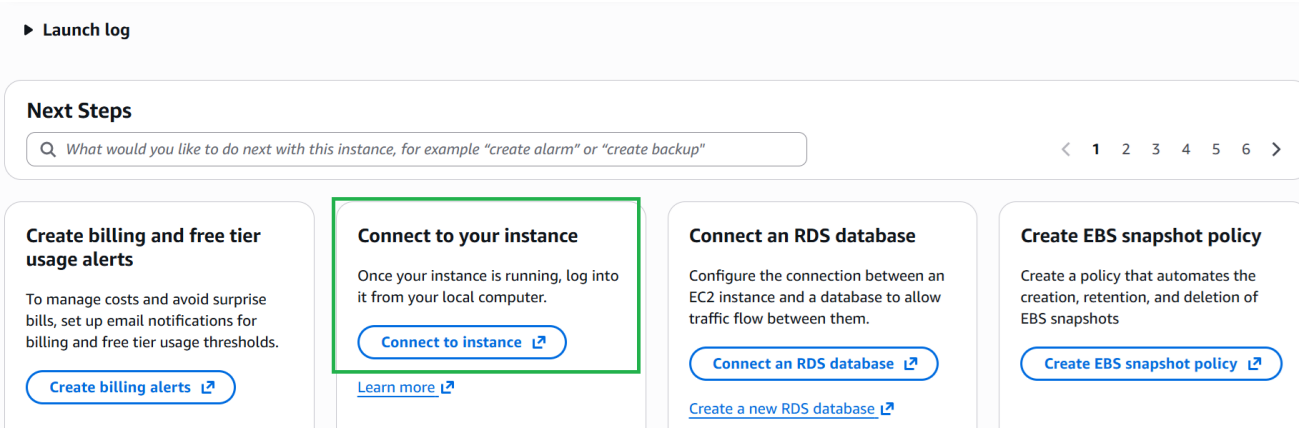
Upload a file with your user data or enter it in the field.

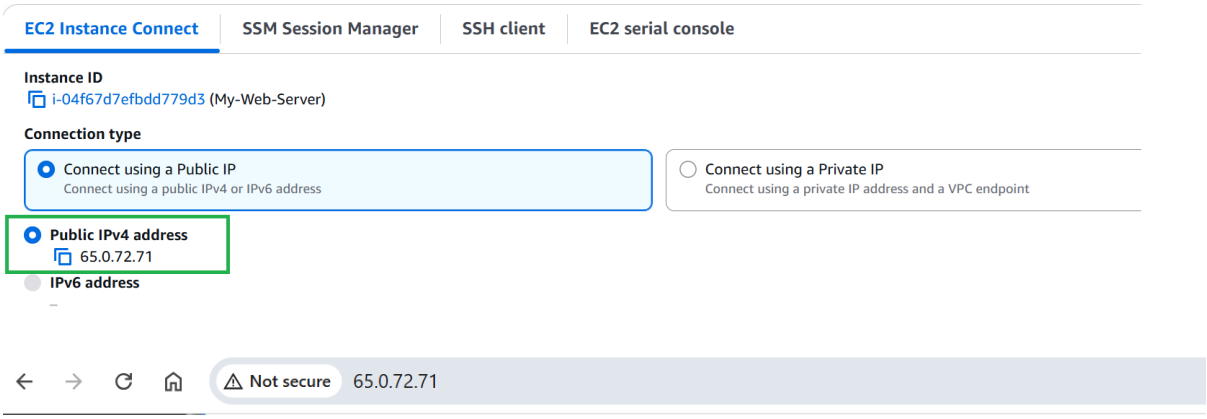
Choose file

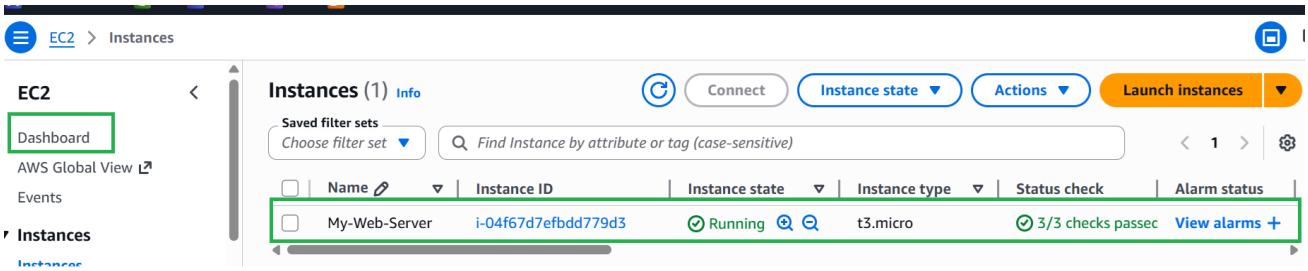
```
#!/bin/bash
dnf update -y
dnf install -y httpd
systemctl start httpd
systemctl enable httpd
echo "<h1>Hello from my EC2 Instance!</h1>" >
/var/www/html/index.html
```

```
#!/bin/bash
dnf update -y
dnf install -y httpd
systemctl start httpd
systemctl enable httpd
echo "<h1>Hello from my EC2 Instance!</h1>" > /var/www/html/index.html
```

*This will automatically install a web server and create a "Hello World" page for you.*




## Summary of what to click:

1. **Name:** Give it a name.
2. **AMI:** Amazon Linux 2023 (Default).
3. **Instance Type:** t3.micro.

4. **Key Pair:** Create a new one, download the `.pem`.
5. **Security Group:** Allow SSH (My IP) and HTTP (Anywhere).
6. **Storage:** 20 GB.
7. **Launch:** Click the big orange button!

### Quick Status Check

1. Go back to the **Instances** dashboard.
2. Wait for **Instance State** to say `Running`.
3. Wait for **Status Check** to say `2/2 checks passed`.
4. Copy the **Public IPv4 address**—this is your server's "phone number" on the internet.

Since  EC2 instance is fresh and you've been working extensively with **Java Spring Boot**, **Docker**, and **Angular**, the best "demo" is to turn this instance into a functional web server.

Here are three great ways to verify your setup is working, ranging from a simple "Hello World" to a full Docker deployment.

## Option 1: The "Instant Win" (Standard Web Server)

This verifies that your **Security Group** (HTTP Port 80) is configured correctly. Run these commands in your EC2 terminal:

1. **Install Apache:** `sudo dnf install -y httpd`
2. **Start it:** `sudo systemctl start httpd`
3. **Create a page:**

Bash
```
echo "<h1>EC2 Demo: Server is UP!</h1><p>Running on Amazon Linux
2023</p>" | sudo tee /var/www/html/index.html
```

4. **Test:** Copy your **Public IPv4 address** from the AWS Console and paste it into your browser. You should see your custom message.

## Option 2: The Docker Demo (Modern Setup)

Since you've been practicing containerization, let's see if we can get Docker running on this instance. This is perfect for your Spring Boot or Nginx images.

1. **Install Docker:**

Bash
```
sudo dnf install -y docker
sudo systemctl start docker
sudo usermod -a -G docker ec2-user
```

2. **Log out and log back in** (to apply the group change).
3. **Run a demo Nginx container:**

Bash
```
docker run -d -p 80:80 --name web-demo nginx
```

4. **Test:** Refresh your browser. You'll now see the "Welcome to nginx!" page being served from a container.

## Option 3: Check Your Java Environment

If you plan to deploy your **Spring Boot** applications here, check if the environment is ready:

- **Check Java:** `java -version`
- **Install OpenJDK 17/21 (if missing):**

Bash
```
sudo dnf install java-21-amazon-corretto-devel -y
```

## Troubleshooting Tip

If you paste the IP address into your browser and it **times out**:

1. Go back to the **EC2 Dashboard** -> **Instances**.
2. Click on your **Instance ID** -> **Security** tab.
3. Click the **Security Group** link (e.g., `launch-wizard-1`).
4. Ensure there is an **Inbound Rule** for **HTTP (Port 80)** with the source **0.0.0.0/0**.